Local Proof Scopes and Interpretation Obstructions for iPRA

A 668-byte checker and a non-bi-interpretability theorem for tied-seed recursion

Prepared with ChatGPT

November 10, 2026

Abstract

This investigation produces a substantive negative result and a complete implementation checkpoint. The requested significant improvement of the positive axiomatization remains unresolved.

The main new result is that ordinary tied-seed arithmetic T_B, with its computation equations, coverage, separation, and literal-equation induction, has *no finite-dimensional bi-interpretation with* iPRA. Arbitrary formula-defined domains, operation graphs, and quotient equality are allowed. The proof combines a finite-profile invariant for terms, finite witness extraction from existential-free axioms, and the source-side isomorphism required by bi-interpretability. It applies more generally to existential-free extensions in the same numerical language whose axioms hold in the standard model. It does not rule out equiconsistency alone or arbitrary one-way interpretations.

The finite-profile invariant also excludes numerical equality testing, minimum, and truncated subtraction as tied-seed terms. Separately, a weaker one-coordinate bound rules out a term-defined two-sided unit on an infinite quotient of any finite-dimensional domain, even after adjoining bounded-valued numerical operations. A zero-start counter recursor is available; adjoining an unbounded zero selector restores initialized primitive recursion by an explicit numerical compiler.

The delivered standalone checker retains the independent initializer and the inherited whole-domain bi-interpretation with iPRA. Local eigenparameter scopes replace its mutable global allocator. The whole program has **668 normalized bytes, 284 tokens, and 323 AST nodes**, with a **391-byte gzip-9** encoding. The inspected corpus's separate minima are **672 bytes, 277 tokens, and 311 AST nodes**. Only the byte coordinate improves, by four bytes. This is an implementation transformation, not a new arithmetic or logical basis.

The package includes full primitive rules, attributed inherited bridge proofs, executable certificates, negative controls, source hashes, and reproducible measurements. The arguments are written mathematical proofs, not Lean or Rocq formalizations.

Status relative to the request. A four-byte implementation gain and new obstructions do not amount to a new smallest axiomatization or a simultaneous improvement of the audited complexity envelope. That positive research target remains unresolved in this report.

Contents

1 The comparison that a new construction must meet 3

2 The complete axiomatization retained by the checker 4

3 Why the retained theory is bi-interpretable with iPRA 5

4 Local eigenparameter scope instead of a global allocator 7

5 The complete program and its public boundary 8

6 A stronger obstruction to removing the initializer 10

7 A stronger term invariant: only finite information from other inputs 14

8 An exact selector boundary for the tied-seed recursor 18

9 The obstruction to relational bi-interpretability 20

10 A small countermodel to an attractive logic merger 25

11 What fixed countdown uniqueness actually determines 25

12 Validation and reproducibility 26

13 Further research questions 27

Inherited complete bridge: attribution 28

A Complementary introductions with one local assumption 28

B Internal scalar predecessor bootstrap 33

C From literal-equation induction to full intuitionistic PRA 37

D Pairing, primitive recursion, and internal numerical composites 43

E Arithmetic strength and both ordinary interpretation composites 44

1 The comparison that a new construction must meet

The corpus is pinned to ProveIt commit

`3fb35a5f2c0cb506180bede7ee3b9b5f93d4bc9d`.

All sixteen ZIP archives directly under `docs/drafts` were read. Recursively including their nested provenance archives gives 22 distinct archive byte strings and 297 distinct Python source byte strings, after SHA-256 deduplication. Every measured source parses. The independent scan and its content-bound classification are included under `audit/`. Sources were measured without executing proposal code. The pin fixes the comparison; it is not a claim about later repository commits. The source repository is cited in [1].

All otherwise nondominated short supporting programs were inspected before exclusion. Twenty-three such sources are builders, measurers, packagers, reproduction runners, or untrusted certificate translators. Complete historical controls inside `reference/` and `ancillary/` do count. Ignoring those controls would overstate the available improvement.

Complete standalone source	Tokens	AST	Bytes	Raw	gzip-9
Constructed computation, explicit	277	317	684	684	403
Flat computation	278	314	679	679	402
Scalar-successor matched byte control	277	319	679	679	401
Scalar-successor matched explicit control	277	317	684	684	405
Transient-seed structure profile	280	311	705	705	393
Counter-congruence zero-product control	281	322	672	672	397
Observed coordinate envelope	277	311	672	—	—
Local scope parameter, this report	**284**	**323**	**668**	**668**	**391**
Readable local scope implementation	320	363	1426	2354	1023

The byte saving against the incumbent is $4/672 \simeq 0.595\%$. Against that exact same source the new implementation uses three more tokens and one more AST node. Against the corpus envelope it uses seven more tokens and twelve more AST nodes. It is therefore inappropriate to describe it as uniformly smaller. In particular, no aggregate score was chosen after seeing the measurements to conceal the unfavorable coordinates.

The six old programs in the table form the reviewed complete-source Pareto frontier. Exact paths, hashes, and all six original source files are in `audit/frontier.json` and `audit/reference/`. The new checker has SHA-256

`e3a77de47e95c99d27455b88b219a7d906ab862b0267c321722a87eaf2be8d6b`.

1.1 What the size and gzip figures mean

The unchanged measurement convention removes genuine docstrings, parses the entire module, and uses `ast.unparse` followed by one newline as normalized source. It counts all AST nodes and all significant Python tokens, excluding comments, indentation/newline metadata, encoding, and end markers. Imports, initial state, syntax traversal, logical and numerical dispatch, JSON input, and both verdict paths are charged. The audit uses CPython 3.12.14 and zlib 1.3.2; its six baseline results agree exactly with the inherited normalizer.

The requested informational compression measurement is

```
len(gzip.compress(raw_source_bytes, compresslevel=9, mtime=0))
```

It includes the gzip header and trailer, has a zero timestamp and no filename, and compresses the actual whole source file. The exact `code/checker.py.gz` is included. Normalized-source gzip is also recorded separately. The compact sources already equal their normalized text, so those two gzip sizes coincide for them. The readable source retains explanatory comments and has a different raw size. Gzip was not used as an optimization objective or to select the Pareto frontier.

The interpreter and standard JSON library are shared platform costs, as in the supplied drafts. No executable proof input, theorem library, compressed source loader, or hidden axiom table is omitted from the counted checker. Producers and measurements are untrusted utilities. The observed minima are attainable upper bounds on an implementation optimum, not mathematical lower bounds over all possible theories and all languages.

2 The complete axiomatization retained by the checker

The target is full first-order intuitionistic primitive recursive arithmetic: ordinary intuitionistic first-order logic with actual equality and falsity, finite primitive-recursive descriptions and their defining equations, successor boundary laws, and induction restricted to quantifier-free formulas. We do not replace this by the merely equational theorem set or by classical PRA. The distinction between classical and constructive presentations is substantive; see, for example, Avigad [10].

The following native presentation is inherited from the generated-assumption and Choice-DI developments [2, 3]. It is stated explicitly because the small checker must account for logic as well as arithmetic. Its reuse is not claimed as a new axiomatization.

2.1 Numerical syntax and the two computation schemas

There is one numerical sort, with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only the update h binds v; other variables in it are fixed passive parameters. The only numerical computation schemas are

$$I(0, a; v.h) = a, \tag{I0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{IS}$$

Every description is fixed finite syntax. One can equivalently elaborate the binder into a staged first-order signature, introducing i_h only after its body has been described. There is no quantified program variable and no uniformly total evaluator for all primitive-recursive descriptions.

2.2 Formula syntax and every primitive logical rule

For the checker it is convenient to allow the auxiliary closure in formulas:

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The paired variables are distinct. The ordinary readings are

$$\mathsf{Q}_y B \;\mapsto\; B(0) \to \forall y B(y), \qquad \mathsf{D}_{x,y} B \;\mapsto\; \exists x \big(B(x, 0) \to \forall y B(x, y)\big).$$

Contexts Γ are finite reusable lists. The full primitive rule inventory is the following:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \qquad \mathsf{Hyp},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \mathsf{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \mathsf{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \qquad \Gamma, \mathsf{Q}_y B(a,y) \vdash F}{\Gamma \vdash F} \qquad \mathsf{DE},$$

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)} \qquad \mathsf{DI}_{\mathrm{case}},$$

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x,y)} \qquad \mathsf{DI}_{\mathrm{progress}}.$$

Here F is any fixed capture-free native formula template. In $\mathsf{DI}_{\mathrm{progress}}$ the matrix E must be a literal equation. A Q or D formula with an equation below its binder does not qualify. In DE, a is fresh for the retained context, the conclusion, and the fixed matrix parameters. In both introductions, z is fresh for the retained context, the witness, and the fixed matrix parameters. Local assumptions are discharged. All substitutions avoid capture, with bound variables renamed when necessary.

Computation is supplied by the rule

$$\frac{\Gamma, H \vdash A}{\Gamma \vdash A}\ \mathsf{Comp}, \qquad H \text{ an instance of I0 or IS.}$$

The designated contradiction is the native equation $0 = S0$. There are no additional logical or numerical axioms, no primitive reflexivity or explosion rule, no ordinary coverage axiom, no general induction schema, and no ambient first-order proof checker. Structural weakening, exchange, contraction, capture-free substitution, and cut are admissible by external induction on derivations, with freshening. Comp is equivalent to taking I0 and IS as theorem leaves: one direction uses cut, and the other uses Comp followed by Hyp.

An auxiliary $\mathsf{Q}_y B$ is interderivable with $\mathsf{D}_{v,y} B$ for unused v. Case introduction gives one direction; paired elimination followed by the supplied closure assumption gives the other. Uniform matrix congruence then removes nested occurrences. Thus the exported Eq/D language and this convenient auxiliary language have the same interpreted logical force.

3 Why the retained theory is bi-interpretable with iPRA

The full inherited proofs are reproduced, unchanged and attributed, in Appendices A–E. The following account identifies the constructions and both composites; equiconsistency alone would not meet the requested equivalence.

3.1 Logic is recovered before the full arithmetic compiler

Progress introduction with an unused existential coordinate, followed by DE and QE, derives contextual induction for a literal equation. Case introduction similarly derives numerical zero/successor cases for every fixed formula template. Reflexivity follows from a computation equation and transport. In the

scalar numerical basis define

$$C(n,a,b) = I(n,a;v.b), \qquad Z(n) = C(n,S0,0).$$

Then $C(0,a,b) = a$ and $C(Sn,a,b) = b$ by computation. These terms make formula gates and two-way selectors possible.

For example, a gate on an equation is

$$G_z(s = t) \;:=\; (C(z,0,s) = C(z,0,t)),$$

extended recursively below binders. Its zero instance is provable and its successor instance is interderivable with the original formula. By aligning and padding binders, the inherited construction gives a formula selector $M_z(A,B)$ with zero instance A and successor instance B, up to proved interderivability. Consequently one can define implication by $\mathsf{D}_{u,z}M_z(A,B)$ and existential quantification by $\mathsf{D}_{x,y}G_yA(x)$. The same construction gives conjunction, disjunction, and universal quantification, with all their ordinary intuitionistic introduction and elimination rules. It does not appeal to excluded middle for quantified formulas.

The native contradiction also yields explosion: under $0 = S0$, transport through C gives every numerical equation, and structural recursion on fixed formula syntax gives every native formula. The full details, including universal elimination in a context containing the specialized parameter, are in Appendix A.

3.2 The arithmetic dependency order matters

The scalar iterator update receives only its previous state, not its recursion counter. Its equivalence with the general PR language is therefore a theorem, not a notational convention. The inherited proof proceeds as follows:

1. Construct a scalar predecessor term using finite cycles and a factor map; prove $P0 = 0$ and $P(Sn) = n$ by literal-equation induction and numerical cases. Pairing and arbitrary quantifier-free induction are not inputs to this construction.
2. Construct truncated subtraction and a Boolean characteristic of actual numerical equality. The proof uses a residual equation, not an assumed equality decision procedure.
3. Translate each fixed quantifier-free formula into its Boolean characteristic. Equation induction on that characteristic yields the corresponding quantifier-free induction instance.
4. Only then construct pairing and projections, and compile each fixed general primitive-recursive description into scalar iteration.

These identities hold internally at arbitrary elements, including in nonstandard models. Numerical experiments would not establish this fact. Each induction keeps passive parameters fixed; later diagonal substitutions are made only into previously proved identities.

3.3 The two interpretations and their composites

Let T_I be ordinary intuitionistic first-order equality logic in the staged iterator signature, with I0, IS, literal-equation induction, coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, and the explicit separation sentence $(0 = S0) \to \bot$. Its ordinary logic has the usual hypothesis and equality rules, the introduction/elimination rules for $\to, \wedge, \vee, \forall, \exists$, and explosion, with their standard eigenvariable restrictions. These describe the comparison theory; they are not extra uncounted primitives of the native checker.

Interpret native equality by actual equality, Q and D by the formulas above, and the domain by $x = x$. Every native rule is sound in T_I: case introduction uses coverage, and progress introduction uses equation induction. Conversely, the reconstructed logical interface, coverage, equation induction, and explosion recover T_I in the native theory. The falsity bridge must be explicit in T_I: its positive numerical equations alone admit a singleton model.

Interpret each scalar iterator in iPRA by its fixed PR definition. In the other direction use the scalar compiler for each fixed PR description. Equation induction proves uniqueness of a prescribed recurrence with a fixed initializer. That uniqueness supplies coherence under substitution and proves that each interpreted-back function agrees with its original on the whole domain. Both domain composites are therefore witnessed by the identity graph $x = y$. The gate and selector proofs give the formula composites by structural induction. This is a whole-domain bi-interpretation with actual equality, and hence an equiconsistency result. The new scope mechanism leaves these interpretations and their composites unchanged.

4 Local eigenparameter scope instead of a global allocator

The inherited program decreases one global integer whenever any subproof allocates a fresh numerical name. Proving a major premise therefore permanently advances the allocator used by the subsequent continuation, even though the major's local hypotheses and eigenparameters have been discharged. That global history is unnecessary.

The new program passes the current floor k to each checking call. The substitution routine is a local closure over that call's floor. A recursive major-premise proof receives the caller's k by value; its internal allocations cannot change the caller's binding. When it returns, the continuation may reuse the next local name. The input cursor still consumes a global certificate tape. This is a change to eigenparameter scope, not a claim that the whole program is pure or mutation-free.

Theorem 4.1 (Local scope preserves the represented calculus)**.** *With the same sorted tree decoding and numerical/logical rule correspondence as the inherited generated-assumption kernel, the local scope kernel is sound for that calculus. Every finite canonical derivation of the calculus has a local-scope certificate. In particular, the represented closed native theorem set and the bi-interpretations in Section 3 are unchanged.*

Proof. At entry to a call $c(a, g, k)$, the scope contains the rigid integer labels $k, k + 1, \ldots, -1$. The initial call has $k = 0$ and empty context. Each continuation appends exactly one context entry and decreases its floor by one. Hence, for canonical traversals, $k = -|g|$ at every call entry. Administrative truth assumptions are included in this count.

Induct on a finite successful checking trace. The entry traversal checks the goal at its current scope before allocating a new name. The major-premise call, when present, checks its input formula at the same scope. Its conclusion therefore has no free occurrence of a name first allocated inside that subproof. Context extension uses a new list, so none of its local assumptions escape either. By the induction hypothesis it supplies precisely the available major needed by the caller. The caller may now allocate its own fresh $k - 1$; the discharged side proof places no freshness requirement on this independent binding event.

All local openings use the inherited traversal and depth increments. After the local decrement, the closure sees the extended scope, so opening the successor goal and the new hypothesis has the same rule meaning as before. Equality transport, closure use, paired rules, and the generated arithmetic assumption are consequently justified by the same local rule-correspondence argument. The hypothesis branch has its usual meaning. This proves soundness by induction on the trace.

For completeness, start with a finite named derivation and freshen its binders. At a branch with d context entries, label active eigenparameters by the appropriate members of $-1, \ldots, -d$. Serialize a major proof with this same environment and discard its private extensions on return. Serialize the continuation using $-(d+1)$ for its new name. The inherited encoding of each rule and computation instance is otherwise unchanged. Induction on the named derivation constructs a local certificate. This also gives an effective alpha-translation to a globally fresh derivation: assign a distinct global label to each binding event, retain ancestor mappings, and discard a major subproof's private map on return. □

Generated arithmetic assumptions need one inherited qualification. The common continuation can open a computation packet as $H(w)$ at its newly allocated parameter. The required property is a provable zero instance $H(0)$, rather than $\forall w H(w)$. If w is fresh for Γ, A, the rule

$$\frac{\Gamma \vdash H(0) \qquad \Gamma, H(w) \vdash A}{\Gamma \vdash A}$$

follows by substituting zero in the second premise and cutting. The eager recursive-image validation and structural zero-completion argument are inherited from the generated-assumption source. The local scope proof changes no part of that argument. When renaming such a packet, its generated hypothesis is interpreted at the post-allocation extended scope.

4.1 What is and is not preserved at the byte level

Two sibling subproofs can reuse the same negative raw label in the new representation while denoting distinct eigenparameters. They cannot do that under the old monotonically global allocation schedule. Conversely, old packets can mention a numerically allocated name from a finished side proof as a later passive parameter. The new format has only its active lexical scope. Such a parameter can be eliminated by substitution and the proof reserialized; it does not add a theorem.

Consequently the claim is preservation of the calculus under scope-respecting alpha-renaming, not equality of the two programs' accepted byte strings. The delivered producer starts with named proof trees and emits the new convention. It is not a claimed universal raw-certificate converter.

4.2 The fresh-witness check is essential

Consider

$$\mathsf{D}_{x,y}\big(I(y, 0; v.Sx) = y\big).$$

Its zero instance is true. Its ordinary interpretation would require all positive numbers to equal one fixed Sx, so the formula is false in the standard model. If an introduction were allowed to choose x as the not-yet-fresh step parameter z, the purported step

$$I(Sz, 0; v.Sz) = Sz$$

would be a genuine computation. This gives an explicit unsoundness regression for deleting the lower scope check. The supplied forged packet is rejected by both implementations and accepted by a mutant with that guard removed, in ordinary and optimized Python.

5 The complete program and its public boundary

The following is the entire measured source, including parser and verdict driver. The retained compact list test and arity constants are inherited implementation choices; their spelling is not the new idea.

```
import json

def c(a, g, k=0):
    i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if a * 0 == [] else
     a if k <= a < d else d % 2 and i(t, d=1)
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g, k)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else t]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)], k)
try:
    q = json.load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The input is a JSON array whose last item is the requested closed formula. Earlier cells are four-field packets $[m, s, u, t]$, consumed from right to left. A hypothesis closes without consuming a packet. Major proofs precede their continuation in consumption order. An unread prefix is inert, matching the old public contract. Acceptance prints `ACCEPT` and exits zero. Exceptions print `REJECT`, exit nonzero, and may include a traceback. No assertion can disappear under Python's `-O` option.

The tree representation is sorted by context:

Array arity	Formula meaning	Numerical meaning
0	Truth $0 = 0$	Zero
1	Closure Q	Successor
2	Equation	Zero with checked annotations
3	Paired closure D	Scalar iterator

For arities one, two, and three, traversal increases the depth by two, one, and four. Formula depth is even and numerical depth is odd. Protected slot pairs are aliases, except that the two bound variables of D have their distinct numerical meanings. The iterator's two body slots alias its one state variable. Extra slots under its counter and initializer are dummy zeros. Numerical binary arrays decode to zero; they do not introduce a pairing operation or new equation sort. All their annotations are still traversed.

A scalar satisfying $k \leq a < d$ is retained. An exposed numerical leaf is replaced by the supplied image, which is itself checked at numerical root. An exposed formula leaf becomes truth. Missing used images and unsupported used data raise. Canonical certificates use integers and arrays; the unchanged decoder also permits harmless numeric and inert-field aliases. This report does not claim that it has introduced a strict canonical parser. In particular, a label outside the rigid range inside an explicit template can denote an exposed substitution slot; it is not an undeclared free parameter.

For a truthy major, the first two matching patterns are equality transport and closure use. Otherwise

a unary or binary major supplies only administrative truth, and a ternary major supplies the opened closure for paired elimination. A falsy major with truthy final field introduces a paired formula at that witness. The mode distinguishes case from permitted literal-equation progress. A falsy final field generates a computation equation. The eager inner traversal of the recursive image is retained even for a constant update: deleting it would let a malformed counter masquerade as a successor.

The annotated implementation in `code/readable_checker.py` gives the same control flow using descriptive names and explicit branches. The exact inherited global-format description is supplied as provenance; its global-allocation paragraph is replaced by Section 4. Physical recursion and memory limits can reject a sufficiently large valid certificate. Completeness, as for the old programs, is a mathematical unbounded-resource statement.

6 A stronger obstruction to removing the initializer

The supplied transient-seed draft proves that every term of a particular initializer-tied recursion language is bounded by a function of at most one input coordinate. It consequently excludes ordinary addition and pairing *as numerical terms*. Its final scope paragraph expressly leaves open changed domains, finite tuples, and quotient interpretations. The present section strengthens that conclusion: arbitrary finite tuples and arbitrary quotient equality still do not help when the interpreted operation has one fixed tuple of term realizers. The gap left by this particular argument is genuinely relational interpretation, rather than merely a different finite-coordinate encoding. Section 9 later excludes relational bi-interpretations for a specified ordinary tied-seed theory by an additional proof-theoretic argument.

The inherited result is in `ipra_transient_seed_2026-11-10.zip`, member `theory/obstruction.tex`, section "A bounded-coordinate obstruction for a tied-seed recursor," especially its "Exact logical scope" subsection. The counter-congruence draft reproduces the same invariant in `theory/obstructions.tex`, section "A more promising seedless operator still has only one unbounded input." Both archives are direct members of the pinned ProveIt snapshot `3fb35a5f2c0cb506180bede7ee3b9b5f93d4bc9d`. The one-coordinate invariant is inherited; the finite-power quotient argument below is the strengthening relative to those inspected drafts. No literature-wide priority claim is made. The broader class of functions bounded in one coordinate is conventionally called the class of *almost unary functions*; see Pinsker [9], Section 1.1.

6.1 The exact numerical grammar being obstructed

Terms are generated by

$$t ::= x \mid 0 \mid B(n; u, v.h),$$

where only the update body h binds the distinct variables u, v. Updates are fixed finite earlier terms, with arbitrary passive numerical parameters. Put

$$Sa := B(0; u, v.a)$$

when u, v do not occur in a. The proposed computation schemas are

$$B(0; u, v.h) = S\big(h[0/u, 0/v]\big), \tag{1}$$

$$B(Sn; u, v.h) = h[n/u, B(n; u, v.h)/v]. \tag{2}$$

Thus the initial state is tied to the same update used in the recursive step; there is no independent initializer field. For each fixed body, the standard natural-number interpretation is

$$B_h(0, \vec{p}) = 1 + h(0, 0, \vec{p}), \qquad B_h(m+1, \vec{p}) = h(m, B_h(m, \vec{p}), \vec{p}). \tag{$\mathrm{B}_{\mathbb{N}}$}$$

Structural recursion on descriptions, followed by numerical primitive recursion, defines this interpretation. In particular the abbreviation S denotes ordinary successor. No uniform evaluator for all descriptions is postulated.

Lemma 6.1 (Inherited coordinate bound, with proof)**.** *For every fixed B-term $t(x_1, \dots, x_r)$, either its standard values are bounded by a constant, or there are an index j and a nondecreasing unary primitive-recursive function F such that*

$$t(x_1, \dots, x_r) \le F(x_j) \quad \textit{for all } (x_1, \dots, x_r) \in \mathbb{N}^r. \tag{CB}$$

The coordinate and bound can be obtained from the finite syntax. The bounding function is not asserted to be a term of the B language.

Proof. Use external structural induction on terms, treating the two bound variables of an update as independent input coordinates when applying the induction hypothesis to its body. Variables and zero satisfy the claim. Consider $B(n; u, v.h)$, renaming u, v away from the free variables of n and the passive parameters.

If h is globally bounded by K, the initial state is at most $K + 1$ and every later state is at most K. If its controlling coordinate is a passive parameter p_j, with $h(u, v, \vec{p}) \le H(p_j)$, every state is at most $1 + H(p_j)$. These bounds are independent of the counter value.

If its controlling coordinate is u, then $h(u, v, \vec{p}) \le H(u)$ for a nondecreasing H. Equation ($B_{\mathbb{N}}$) bounds the state at counter m by $1 + H(m)$: this includes the base state, and the positive state at $m + 1$ is at most $H(m)$.

Finally, if its controlling coordinate is v, set

$$K(0) = 1 + H(0), \qquad K(m + 1) = \max\{K(m), H(K(m))\}.$$

This is a nondecreasing primitive-recursive function. External induction on the standard counter proves $B_h(m, \vec{p}) \le K(m)$, uniformly in all passive parameters: the base is immediate, and the next state is at most $H(B_h(m, \vec{p})) \le H(K(m)) \le K(m + 1)$.

In the last two cases write the nondecreasing counter bound as K. Apply the structural induction hypothesis to the actual counter term n. A constant bound $n \le c$ gives the constant bound $K(c)$. A bound $n(\vec{x}) \le N(x_j)$ gives $K(N(x_j))$. Composition preserves monotonicity and primitive recursiveness. All possible controlling coordinates of h have now been covered. □

This is a theorem about standard term operations, not an object-language induction principle. Adding axioms that are sound in this standard algebra cannot change those term operations or invalidate the bound.

6.2 The finite-power quotient theorem

The next theorem uses only the conclusion of Lemma 6.1; it applies to any algebra on $\mathbb{N}$ whose term operations have that property. Its sets and equivalence relations need not be definable.

Theorem 6.2 (No infinite quotient with a term-defined two-sided unit)**.** *Let $d \ge 1$, let $D \subseteq \mathbb{N}^d$, let $\sim$ be any equivalence relation on D, and let $e \in D$. Suppose that a map*

$$\mu = (\mu_1, \dots, \mu_d) : D \times D \longrightarrow D$$

is represented by one fixed tuple of terms in its $2d$ input coordinates, with any fixed numerical parameters, and that

$$\mu(e, x) \sim x \sim \mu(x, e) \qquad (x \in D). \tag{Unit}$$

Then $D/\sim$ is finite.

More precisely, a composition of at most

$$m = \lceil \log_2(d+1) \rceil$$

maps of the forms $x \mapsto \mu(e,x)$ and $x \mapsto \mu(x,e)$ has finite image and sends every element to its own $\sim$-class.

Proof. Define

$$T_0(x) = \mu(e,x), \qquad T_1(x) = \mu(x,e).$$

By closure and (Unit), both maps send D into D and preserve each $\sim$-class. The same holds for every finite composition. Only transitivity of $\sim$ is used here.

For each output coordinate i, choose a bound supplied by Lemma 6.1 for μ_i. A bound controlled by one of the fixed numerical parameters is a constant after those parameters are fixed, so it can be included in the constant case. Mark a coordinate with a constant bound as *terminating*.

Every remaining coordinate has a controlling variable from one of the two input tuples. If

$$\mu_i(x,y) \le F_i(x_j),$$

record an edge $i \longrightarrow j$ labelled 1. In that case

$$(T_0(x))_i \le F_i(e_j), \qquad (T_1(x))_i \le F_i(x_j). \tag{First input}$$

If instead

$$\mu_i(x,y) \le F_i(y_j),$$

record an edge $i \longrightarrow j$ labelled 0. Then

$$(T_1(x))_i \le F_i(e_j), \qquad (T_0(x))_i \le F_i(x_j). \tag{Second input}$$

Thus at every nonterminating coordinate there is exactly one label that can leave dependence on the variable input in the selected bound, and exactly one next coordinate to which that dependence passes. Choosing the other label gives a constant bound immediately.

Starting at a fixed coordinate i, this deterministic labelled graph admits at most one binary word of length m along which a dependence bound survives all m steps. A path meeting a terminating coordinate does not admit such a word. Consequently at most d of the 2^m binary words are admitted from any of the d starting coordinates. Since $2^m > d$, choose a word $w = b_1 \cdots b_m$ admitted from none of them, and put

$$T_w = T_{b_1} \circ T_{b_2} \circ \cdots \circ T_{b_m}. \tag{Composition order}$$

For clarity, a coordinate bound is traced from the *outermost* operation to the innermost, so the word in (Composition order) is read from b_1 to b_m. For instance, if the selected bound for the outer coordinate permits b_1 and leads from i to j, then

$$(T_w(x))_i \le F_i((T_{b_2} \circ \cdots \circ T_{b_m}(x))_j).$$

The choice of w ensures that this chain reaches a terminating bound or a mismatched edge label no later than the last letter. The bound at that stage is a fixed constant. Composing outward with the preceding nondecreasing unary functions yields a fixed constant C_i independent of x. This verifies the bound in the specified composition order; no commutation or rearrangement of the maps T_0, T_1 is assumed.

Therefore

$$T_w(D) \subseteq D \cap \prod_{i=1}^{d} \{0, 1, \ldots, C_i\},$$

which is finite. Yet $T_w(x) \sim x$ for every $x \in D$. Every $\sim$-class has a representative in this finite image, so the quotient is finite. □

Remark 6.3 (Very little algebra is needed). The theorem does not use associativity or commutativity of μ. It does not even require μ to respect $\sim$ in both inputs; the two displayed unit identities and closure suffice. In applications where μ interprets a monoid operation on a quotient, compatibility is available but unnecessary to this argument. If $d = 0$, the ambient power is a singleton and finiteness is immediate.

6.3 Consequence for interpretations of arithmetic

Corollary 6.4 (Finite dimension and quotient equality do not repair a term compiler)**.** *Let T be a theory satisfied by the standard B-algebra described in* ($\mathrm{B}_{\mathbb{N}}$)*. There is no interpretation of full first-order* iPRA *in T having all the following properties:*

1. *the interpreted numerical domain is a nonempty definable subset D of one finite power of the source domain;*
2. *interpreted equality is any definable equivalence relation on D;*
3. *interpreted addition has one fixed tuple of B-term realizers mapping $D \times D$ into D.*

The claim allows fixed parameters and any interpretation of successor consistent with the target axioms. In particular, there is no bi-interpretation meeting these restrictions.

Proof. Apply the assumed interpretation in the standard B-model of T. Choose a representative e of interpreted zero. The translated addition unit laws give (Unit). By Theorem 6.2, the interpreted numerical quotient is finite. But the translated target axioms give an injective successor map on that quotient whose image omits zero. An injective self-map of a finite nonempty set is surjective, a contradiction. □

The qualifier concerning the standard B-model is essential. A theory whose additional axioms deliberately exclude that model is not ruled out solely by an analysis of its standard term functions. The natural candidate based on the displayed computations and rules sound over $\mathbb{N}$ does have this model.

Likewise, the restriction on the interpreted operation is essential. An ordinary first-order interpretation may define an operation by a formula with a unique output modulo the interpreted equality, rather than by a fixed tuple of source terms. Such graphs remain outside this theorem. Even a finite definition by cases can choose different controlling input coordinates in different cases. For example, the operation $\max(x, y)$ on $\mathbb{N}$ has identity zero and is defined by cases selecting one of its arguments, whereas no single coordinate bounds it. This example explains the scope limitation; it is not a claim that order or maximum is definable in the B-algebra.

The logarithmic bound in Theorem 6.2 counts layers of unit multiplication in a metatheoretic construction. It is not a bound on expanded numerical term size, evaluation time, or proof-checker source size. The theorem strengthens a negative result for one proposed smaller arithmetic basis. It does not provide the smaller, complete bi-interpretable axiomatization requested in the task.

7 A stronger term invariant: only finite information from other inputs

The coordinate bound leaves open whether the tied-seed language could compute a bounded numerical comparison. This section closes that particular route. The gate supplied by the current drafts does not suffice to construct a numerical equality test, minimum, or truncated subtraction. A stronger semantic invariant excludes all three.

Definition 7.1 (One numerical input and finite control)**.** A function $f : \mathbb{N}^r \to \mathbb{N}$, with $r \geq 1$, has the finite control property if there are an active coordinate i, finite nonempty sets A_j and unary maps $\alpha_j : \mathbb{N} \to A_j$ for $j \neq i$, and unary functions $F_a : \mathbb{N} \to \mathbb{N}$ indexed by $a \in \prod_{j \neq i} A_j$, such that

$$f(x_1, \ldots, x_r) = F_{(\alpha_j(x_j))_{j \neq i}}(x_i). \tag{FC}$$

Thus the value may depend without a bound on one coordinate, while each other coordinate supplies only a member of a fixed finite set. For primitive-recursive functions, all displayed maps will be chosen primitive recursive, using finite encodings of the sets A_j.

A constant function satisfies the definition with any active coordinate. Information about the active coordinate may also determine a finite choice, since that choice can be absorbed into the unary function F_a. The definition imposes no restriction at arity one. Throughout the constructive argument, finite ranges may be replaced by the specified finite codomains; no decision procedure for exact attained ranges is assumed.

Lemma 7.2 (Finite-valued functions and composition)**.** *1. If f has the finite control property and ρ is unary with finite range, then $\rho \circ f$ factors through finite-valued unary maps of* all *input coordinates.*

2. The functions with the finite control property contain constants and projections and are closed under composition.

Both assertions preserve primitive recursiveness of the displayed maps.

Proof. For the first assertion, use (FC) and attach to the active coordinate the finite feature

$$\beta_i(x) = \big(\rho(F_a(x))\big)_a.$$

There are finitely many a, and each entry lies in the finite range of ρ. Together with the existing $\alpha_j(x_j)$, this determines $\rho(f(\vec{x}))$.

For composition write $f(g_1(\vec{x}), \ldots, g_r(\vec{x}))$, where f has active input i and g_i has active input k. Every other g_j enters f only through its finite-valued feature $\alpha_j \circ g_j$. By the first assertion these features are determined by finite unary features of all coordinates of $\vec{x}$. Combine their features on coordinates other than k with the finite controls of g_i. Once those combined controls are fixed, the whole expression is one unary function of x_k: features depending on x_k are evaluated inside that unary function. There are only finitely many combined controls, giving the required form. All vector formation, finite selection, and composition here are primitive recursive when the original maps are. □

Theorem 7.3 (Finite control for every tied-seed term)**.** *Every standard numerical function of positive arity represented by a B-term in the grammar of Section 6 has the finite control property. Nullary terms are constants and satisfy the same conclusion after an unused input is added. A finite representation of the form* (FC)*, with primitive-recursive feature maps and profiles, can be constructed by induction on its syntax.*

Proof. Variables and zero are covered by Lemma 7.2. It is enough to show that the operation $B_h(n, \vec{p})$ preserves the property when its fixed update $h(u, v, \vec{p})$ has it. The actual counter term is then substituted using the composition lemma.

The active input of h is its previous state. Write

$$h(u, v, \vec{p}) = F_{\alpha(u),\gamma(\vec{p})}(v),$$

where $\gamma(\vec{p})$ is a tuple of finite unary features, one for each passive coordinate. For each of its finitely many values c, define one unary sequence by

$$R_c(0) = 1 + F_{\alpha(0),c}(0), \qquad R_c(m+1) = F_{\alpha(m),c}(R_c(m)).$$

These are primitive-recursive unary functions. Directly from the standard defining equations,

$$B_h(n, \vec{p}) = R_{\gamma(\vec{p})}(n).$$

This has active coordinate n and finite controls on the passive inputs.

The active input of h is its old counter. Write

$$h(u, v, \vec{p}) = F_{\beta(v),\gamma(\vec{p})}(u).$$

For each finite passive profile c, define

$$R_c(0) = 1 + F_{\beta(0),c}(0), \qquad R_c(m+1) = F_{\beta(R_c(m)),c}(m).$$

Again $B_h(n, \vec{p}) = R_{\gamma(\vec{p})}(n)$, of the required form.

The active input of h is a passive parameter p. Write the remaining passive parameters as $\vec{q}$ and put

$$h(u, v, p, \vec{q}) = F_{\alpha(u),\beta(v),\gamma(\vec{q})}(p).$$

The ranges of α, β, and γ are finite. Form one finite catalogue of unary functions $V_j(p)$ containing all profiles $F_{a,b,c}(p)$ and also every possible initial profile

$$V_{0,c}(p) = 1 + F_{\alpha(0),\beta(0),c}(p).$$

Give these occurrences separate indices if their extensional functions happen to coincide. Denote the finite index set by J. Every state of the recursion is the value at p of one function in this catalogue.

The only observation of a previous state used by the update is its β-value. Record all such observations at once in the finite unary feature

$$\kappa(p) = \big(\beta(V_j(p))\big)_{j \in J}.$$

For each passive control value c and each possible finite vector k, define a finite-state process. Its initial state is the catalogue index of $V_{0,c}$. At counter m, if its present index is j, its next index is the catalogue occurrence of

$$F_{\alpha(m),k_j,c}.$$

Let $\theta_{c,k}(n) \in J$ be the index after n transitions. This is a finite-valued primitive-recursive unary function of n. It is defined for all possible vectors k, including ones not realized by any p; only

the realized vectors will be used in the equality below. Pack the finitely many processes into one finite-valued unary feature

$$\eta(n) = \big(\theta_{c,k}(n)\big)_{c,k}.$$

Induction on the standard counter in the metatheory gives

$$B_h(n, p, \vec{q}) = V_{\eta(n)_{\gamma(\vec{q}),\kappa(p)}}(p). \qquad \text{(Catalogue evaluation)}$$

The initial case selects $V_{0,c}$; the transition uses exactly the recorded observation $\beta(V_j(p)) = \kappa(p)_j$.

For each fixed e in the finite product codomain chosen for η and each fixed c, define the unary profile

$$H_{e,c}(p) = V_{e_{c,\kappa(p)}}(p).$$

Equation (Catalogue evaluation) becomes

$$B_h(n, p, \vec{q}) = H_{\eta(n),\gamma(\vec{q})}(p).$$

This has active coordinate p, with finite controls on n and each remaining passive input. It proves the last case. The construction is finite and primitive recursive at every stage, completing the syntax induction. □

The numerical bounds of Lemma 6.1 are also an immediate consequence of this stronger representation: take the maximum of the finitely many unary profiles and then its running maximum. Its new content is that the other inputs supply only finitely many distinct profiles, rather than arbitrarily complicated bounded information.

Corollary 7.4 (No numerical equality test, minimum, or variable countdown)**.** *None of the following binary functions is represented by a B-term:*

$$\mathrm{eq}(a, b) = \begin{cases} 1, & a = b, \\ 0, & a \neq b, \end{cases} \qquad \min(a, b), \qquad a \dot{-} b = \max(a - b, 0).$$

The same exclusion holds for the complementary numerical equality test.

Proof. For a binary function with active first coordinate, fixing the second coordinate produces at most finitely many unary functions of the first. If the second coordinate is active, the reverse collection of unary profiles is finite. Each displayed function has infinitely many distinct profiles in both directions.

For numerical equality, fixing one input at b gives the characteristic function of the singleton $\{b\}$ in the other input. For minimum, the functions $x \mapsto \min(x, b)$ are distinct for distinct b, as is seen by evaluating at an x at least as large as both candidate values. For truncated subtraction, distinct fixed second arguments give distinct functions by evaluating the first argument above both; distinct fixed first arguments give distinct functions already at second argument zero. The finite-profile conclusion of Theorem 7.3 is therefore impossible in either orientation. □

7.1 The exact consequence for the proposed relational route

The current B grammar does define ordinary predecessor and a one-sided gate. Write

$$D(n, a) = B(n; u, v.a), \quad W(n) = B(n; u, v.u), \quad P(n) = W(D(n, n)), \quad Z(n) = D(n, 0).$$

Their computation equations give $D(0,a) = Sa$, $D(Sn,a) = a$, $P0 = 0$, $P(Sn) = n$, $Z0 = 1$, and $Z(Sn) = 0$. With equation induction, the gate

$$G(n,a) = P\big(P(B(a;u,v.D(Z(n),v)))\big)$$

satisfies $G(0,a) = 0$ and $G(Sn,a) = a$. At zero its internal update is the identity and its state is constantly 1; at a positive control its update is successor and its state is S^2a.

For completeness, both gate identities are internal consequences of the B computation schemas, ordinary equality reasoning, and literal-equation induction. For the zero control, induct directly on

$$B(a;u,v.D(Z(0),v)) = 1.$$

Its base is $S(D(1,0)) = S0 = 1$; its successor computation applies $D(1,\cdot)$, which repeats the previous state. For the positive control Sq, induct directly on

$$B(a;u,v.D(Z(Sq),v)) = S^2a.$$

Its base is $S(D(0,0)) = S(S0) = 2$; its successor computation applies $D(0,\cdot)$, hence successor, to the previous state. Keeping these actual update bodies throughout avoids invoking an unmentioned extensionality principle for recursion descriptions. The displayed equations for P then remove the two outer predecessors. All control parameters are fixed during these inductions; no induction on a disjunction or a compound formula is used.

Lemma 7.5 (A positive boundary: arbitrary zero-start counter recursion)**.** *In any such equational presentation, the gate gives a general zero-start counter recursor as a macro:*

$$L(n;u,v.h) := B\big(Sn;r,s.G(r,h[Pr/u,s/v])\big), \qquad \text{(Zero-start macro)}$$

with all new bound variables fresh. It satisfies

$$L(0;u,v.h) = 0, \qquad L(Sn;u,v.h) = h[n/u, L(n;u,v.h)/v].$$

Proof. Let $H(r,s) = G(r,h[Pr/u,s/v])$ be the fixed internal update. The gate equations and $P(Sr) = r$ give

$$H(0,s) = 0, \qquad H(Sr,s) = h[r/u,s/v].$$

Thus $B_H(0) = S(H(0,0)) = 1$ and $B_H(1) = H(0,B_H(0)) = 0$. The latter identity is the base equation for $L(n) = B_H(Sn)$. At a successor,

$$L(Sn) = B_H(S(Sn)) = H(Sn,B_H(Sn)) = h[n/u,L(n)/v].$$

No further induction is used after the gate identities have been proved. □

These statements do not conflict with Theorem 7.3: their macros remain B-terms. In particular, fixed-modulus counting and fixed finite-state controls do not yield a numerical equality test for two unbounded independent inputs.

The theorem does not settle whether every unary primitive-recursive function is represented, because its conclusion imposes no restriction on unary functions. It also does not rule out formula-defined comparison relations: the source already has logical equality $a = b$, which must not be confused with a term returning its numerical characteristic. This term theorem alone leaves quantified operation graphs outside its scope. Section 9 adds a proof-theoretic argument and excludes arbitrary finite-dimensional relational bi-interpretations for the ordinary tied-seed theory with its stated existential-free axiomatization.

8 An exact selector boundary for the tied-seed recursor

The negative result has a precise positive counterpart. A bounded equality characteristic cannot remove the numerical obstruction, whereas an unbounded zero selector gives an immediate compiler for every fixed primitive-recursive description. The latter compiler is a numerical result. It does not by itself establish a smaller complete calculus or discharge the full logical, equality-reflection, induction, and bi-interpretation obligations.

8.1 Bounded-valued primitives preserve the obstruction

Consider the grammar with zero and

$$B(0;u,v.h) = S\big(h(0,0)\big), \qquad B(Sn;u,v.h) = h\big(n,B(n;u,v.h)\big), \qquad Sa := B(0;u,v.a),$$

with the same passive-parameter and capture-free substitution conventions as before. Extend it by any collection of fixed total operations e such that, for each operation separately, there is a constant K_e with

$$e(\vec{x}) \le K_e \quad \text{for every input } \vec{x}.$$

Only finitely many operations occur in any individual term.

Proposition 8.1 (Bounded extensions remain almost unary)**.** *Every term of this enlarged grammar is bounded either by a constant or by a nondecreasing unary primitive-recursive function of one of its free coordinates. In particular, adjoining a primitive equality characteristic with values in* $\{0,1\}$ *does not invalidate the coordinate-bound obstruction.*

Proof. Repeat the structural coordinate-bound argument. A term whose outermost operation is an added e has the constant bound K_e, independently of the bounds of its arguments. This is the only new structural case. At a B-term the update is again either uniformly bounded or bounded in its counter coordinate, state coordinate, or one fixed passive coordinate. The same constant, one-coordinate, and iterated-unary bounds therefore apply. Thus the finite-dimensional quotient obstruction, whenever its interpreted operations are required to have term realizers, applies unchanged. □

The proposition is about term operations in the standard algebra. Further axioms that are sound in that algebra cannot change the bound. It does not exclude interpretations using formula-defined functional graphs.

8.2 One unbounded zero selector restores ordinary recursion

Instead adjoin the numerical operation C with precisely the computation schemas

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b. \tag{C}$$

Allow C in subsequent finite B update descriptions. This syntactic extension is part of the proposed repair and is not implicit. The selector is unbounded: its two output arguments are arbitrary numerical terms. It has no one-coordinate bound, since at control zero its output can grow with a, and at positive control its output can grow independently with b.

Define

$$W(n) := B(n;u,v.u).$$

The two B computations give $W(0) = S0$ and $W(Sn) = n$. Let fixed earlier terms $g(\vec{p})$ and $h(n, z, \vec{p})$ be given. Choose fresh u, v and put

$$\begin{aligned} H(u, v, \vec{p}) &:= C\big(u, g(\vec{p}), h(W(u), v, \vec{p})\big), \\ J(n, \vec{p}) &:= B\big(Sn; u, v.H(u, v, \vec{p})\big). \end{aligned} \qquad \text{(Selector compiler)}$$

Proposition 8.2 (Direct primitive-recursion compiler)**.** *Ordinary equality reasoning and the displayed computation schemas prove*

$$J(0, \vec{p}) = g(\vec{p}), \qquad J(Sn, \vec{p}) = h\big(n, J(n, \vec{p}), \vec{p}\big).$$

No equation induction, predecessor search, equality characteristic, pairing construction, or independent initializer is used in this derivation.

Proof. Suppress the fixed passive parameters. The first actual update discards the tied initial state:

$$\begin{aligned} J(0) &= B(S0; u, v.H) \\ &= H\big(0, B(0; u, v.H)\big) \\ &= C\big(0, g, h(W0, B(0; u, v.H))\big) = g. \end{aligned}$$

At a subsequent update the counter is a displayed successor, so

$$\begin{aligned} J(Sn) &= B(S(Sn); u, v.H) \\ &= H\big(Sn, B(Sn; u, v.H)\big) \\ &= C(Sn, g, h(W(Sn), J(n))) \\ &= h\big(n, J(n)\big). \end{aligned}$$

These are identities with arbitrary numerical parameters, not a computation restricted to standard numerals. The original zero value is $B(0; u, v.H) = Sg$; the compiler deliberately skips it and starts at $B(S0; u, v.H) = g$. □

Zero, successor, and projections are already terms, and composition is substitution. External structural recursion on a fixed primitive-recursive description therefore compiles every primitive-recursive function into this enlarged grammar. Conversely each fixed B, C description has its usual primitive-recursive numerical realization. These two facts establish numerical primitive-recursive completeness. The general logical and interpretation-composite claims require their own proofs and size accounting.

8.3 Why a formula definition alone does not supply this compiler

In ordinary intuitionistic logic, suppose numerical coverage and $0 \neq S0$ are available. Put $Z(n) = B(n; u, v.0)$. Then $Z0 = S0$ and $Z(Sn) = 0$; coverage and these equations prove $z = 0 \vee z \neq 0$. Consequently

$$R_C(z, a, b, w) := (z = 0 \wedge w = a) \vee (z \neq 0 \wedge w = b)$$

is provably total and single-valued. Adjoining a function symbol with this graph is an ordinary definitional extension of that first-order theory.

Two further changes would nevertheless need justification. First, the old B signature has symbols only for its original term updates. Adjoining C by a formula definition does not automatically provide a B symbol for an update containing C, or prove totality and uniqueness of that new recursion graph.

Second, literal-equation induction for new C-terms translates into induction on formulas that need not be literal equations of the old language. Thus closing both the recursor-description scheme and the induction scheme under such definitions is additional mathematics, not a free consequence of conservativity of a single function definition. The explicit positive repair above includes C in the numerical grammar; it makes no claim that the original tied-seed theory already contains it.

9 The obstruction to relational bi-interpretability

The preceding semantic obstructions concerned operation symbols realized by terms. The finite-profile theorem also has a proof-theoretic consequence. For the ordinary tied-seed arithmetic with literal-equation induction, it rules out *every finite-dimensional bi-interpretation* with iPRA, including interpretations with arbitrary formula-defined domains, operation graphs, and quotient equality. The additional ingredient is finite witness extraction from intuitionistic proofs with existential-free axioms.

This conclusion concerns a specified theory, not every possible theory in the same numerical language. Adding positive existence axioms can invalidate the witness-extraction hypothesis. It also does not claim that the tied-seed theory has no one-way interpretation of arithmetic on a complicated domain. The source-side round trip required for bi-interpretability is essential to the final argument.

9.1 The ordinary tied-seed theory has an existential-free presentation

Let T_B use ordinary intuitionistic first-order logic with equality and the staged $0, B$ numerical signature. It has the two computation schemas (1)–(2), numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = S0) \to \bot$, and literal-equation induction. The latter may be given as the universal closures of

$$(E(0,\vec{p}) \wedge \forall n\,(E(n,\vec{p}) \to E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p}), \qquad \text{(EI axiom)}$$

where E is a literal equation. This axiom presentation and the usual contextual induction rule are interderivable by implication and universal introduction and elimination, with the ordinary eigenvariable conditions.

Every axiom in this list except the displayed coverage sentence is existential-free. The predecessor abbreviation P already satisfies

$$P0 = 0, \qquad P(Sy) = y$$

by computation. Consequently coverage is equivalent to

$$\forall x\,(x = 0 \vee x = S(Px)). \qquad \text{(Coverage}')$$

Indeed, a witness y to $x = Sy$ is equal to Px, and the converse uses the explicit witness Px. Neither direction needs induction. Thus T_B has an equivalent existential-free axiomatization in the same numerical language.

For the proof-theoretic argument, equality can be treated as an ordinary binary predicate with universal axioms for reflexivity, symmetry, transitivity, and congruence of every function symbol and atomic predicate symbol. In the numerical B language equality is the only predicate, so its predicate-congruence instances already follow from symmetry and transitivity. All these axioms are existential-free; transport through arbitrary formulas is derived by induction on formula syntax. This avoids invoking a primitive equality rule whose motive could obscure the subformula argument. The

recursion notation is expanded into its staged first-order function symbols before applying first-order cut elimination.

The argument below applies more generally to any ordinary intuitionistic theory T in this numerical language that has an existential-free axiomatization and has the standard B-algebra as a model. An infinite axiom scheme causes no difficulty: a finite proof uses only finitely many axioms. Extra existential-free axioms that are true in the standard model do not evade the result.

9.2 Finite witnesses with an arbitrary matrix

Lemma 9.1 (Finite witness extraction)**.** *Let the language contain a closed term* 0*, and let* Γ *be a finite set of existential-free formulas. If intuitionistic first-order logic proves*

$$\Gamma \vdash \exists z_1 \cdots \exists z_d \, R(\vec{x}, \vec{z}),$$

then there are finitely many tuples of language terms $\vec{t}_1(\vec{x}), \ldots, \vec{t}_M(\vec{x})$ *for which*

$$\Gamma \vdash \bigvee_{\ell=1}^{M} R(\vec{x}, \vec{t}_\ell(\vec{x})). \tag{Witness list}$$

Any additional fixed parameters are retained in the terms. The formula R *may contain arbitrary quantifiers, including existential quantifiers. No uniqueness of witnesses is assumed.*

Proof. Use a cut-free intuitionistic sequent proof. The required cut-elimination theorem is standard; for a published proof see Avigad [11], Theorem 7.6. Equality is handled by the existential-free axioms just specified, so no special equality extension of that theorem is needed.

Starting at the end sequent, trace only the branches carrying the displayed leading existential quantifiers. Keep the finite prefix of witness terms already selected. At a left implication rule retain its first premise proof unchanged and follow its second premise, whose succedent remains the tracked existential formula. At a left disjunction rule follow both premises. The other applicable left and structural rules preserve the tracked succedent in the usual way.

Every antecedent formula reached on these tracked branches is a subformula instance of a formula in Γ, and is therefore existential-free. In particular, no left existential rule can occur there. While at least one of the displayed leading existential quantifiers remains, an initial identity sequent cannot close the branch: its existential succedent is not an antecedent formula. The only applicable right introduction is existential introduction. Record its witness term and continue with the shorter leading prefix. When all d witnesses have been introduced, stop tracing that branch and retain the remaining proof of its matrix instance unchanged. The matrix can be arbitrarily complicated because it is not analyzed.

If a tracked branch instead closes by falsity elimination, fill any unselected witnesses with 0; its resulting matrix instance follows from the same falsity. There are finitely many branches, so this process records finitely many complete tuples $\vec{t}_\ell$.

Now replace each stopped branch by its retained proof of $R(\vec{x}, \vec{t}_\ell)$ followed by the appropriate introductions into the common finite disjunction in (Witness list). Reassemble the tracked left and structural inferences. At an existential introduction that supplied a recorded component, the finite disjunction is already the succedent of the transformed premise, so that introduction is omitted. At left implication inferences the unchanged first premise and the transformed second premise reconstruct the inference. This gives the claimed proof.

There are no newly bound numerical parameters on the tracked branches: left existential introduction is absent, and right universal introduction cannot have the tracked existential succedent. Any extraneous

free parameters of the original finite proof can be replaced uniformly by 0, after renaming local eigenvariables. Thus the witness terms need only the retained parameters and $\vec{x}$. □

This lemma is an application of cut elimination, not a claim that Avigad's paper contains the present arithmetic obstruction. It also does not assert a single uniform term witness. A finite disjunction is exactly what is needed below.

9.3 An arity obstruction for provably total coding relations

Theorem 9.2 (No provable compression from k scalars to d coordinates)**.** *Let T have an existential-free axiomatization and the standard B-algebra as a model. Fix positive integers $k > d$ and any formula $R(\vec{x}, \vec{z})$, where $\vec{x}$ has k scalar coordinates and $\vec{z}$ has d. Suppose T proves*

$$\forall \vec{x}\, \exists \vec{z}\, R(\vec{x}, \vec{z}). \tag{Total}$$

Then it is impossible that, in the standard model, each output tuple $\vec{z}$ occurs with at most one input tuple $\vec{x}$. In particular, T cannot also prove

$$R(\vec{x}, \vec{z}) \wedge R(\vec{x}\,', \vec{z}) \longrightarrow \bigwedge_{i=1}^{k} x_i = x_i'. \tag{Disjoint fibers}$$

The relation may have many outputs for each input.

Proof. Instantiate (Total) at free variables $\vec{x}$ and collect the finitely many axioms used by the proof in an existential-free antecedent. Lemma 9.1 yields a finite list of d-tuples of B-terms $\vec{t}_1, \ldots, \vec{t}_M$ such that, for every input in the standard model, at least one listed tuple is an R-output.

For each scalar coordinate term $t_{\ell j}$, apply Theorem 7.3. Let $q_{\ell j}$ be the finite number of its possible unary profiles. On the grid

$$X_N = \{0, 1, \ldots, N-1\}^k,$$

that term has at most $q_{\ell j} N$ possible values: its active argument has at most N values, and there are at most $q_{\ell j}$ profiles. Constant terms can be treated by adding an unused active argument. Thus the vector $\vec{t}_\ell$ has at most

$$N^d \prod_{j=1}^{d} q_{\ell j}$$

possible values on X_N. The union of the images of all listed tuples has size at most CN^d, where

$$C = \sum_{\ell=1}^{M} \prod_{j=1}^{d} q_{\ell j}$$

is fixed independently of N.

If each output belonged to at most one input, the N^k inputs in X_N would require at least N^k distinct outputs in this union. This is impossible for a sufficiently large standard N, since $k > d$. Taking $N > C$ already suffices. Soundness in the standard model turns any proved instance of (Disjoint fibers) into exactly the forbidden semantic condition. □

Corollary 9.3 (Scalar pairing totality cannot be proved)**.** *In T_B, no formula can be proved to code every pair of source numbers by a single source number with unique decoding. This includes any proposed pairing relation*

$$\pi_1(c) = a \ \wedge \ \pi_2(c) = b$$

using fixed unary B-terms as projections. The conclusion remains true even if the projection relations are arbitrary formulas, provided each code determines at most one pair.

Thus even a future proof of unary primitive-recursive completeness of the B term language would not establish the required pairing totality inside T_B. The obstacle is proof-theoretic as well as term-theoretic.

9.4 Why bi-interpretability supplies forbidden compression

Theorem 9.4 (No finite-dimensional bi-interpretation with iPRA)**.** *Let T have an existential-free axiomatization in the B numerical language and have the standard B-algebra as a model. Then T and full first-order* iPRA *are not bi-interpretable by ordinary finite-dimensional first-order interpretations. The interpreted domains, operation graphs, and quotient equality formulas may be arbitrary.*

The statement is parameter-free, and also applies when fixed parameters are retained uniformly in the interpretation proofs without adding non-existential-free assumptions to T.

Proof. Suppose that I interprets arithmetic $A = \mathsf{iPRA}$ in T, and J interprets T in A. Write

$$I : A \rightsquigarrow T, \qquad J : T \rightsquigarrow A.$$

Let I have dimension d, with numerical domain $D \subseteq \mathbb{N}^d$ and interpreted arithmetic equality E. Let J have dimension e over the arithmetic domain. A zero-dimensional domain has only finitely many tuples, so it cannot participate in the assumed source-side isomorphism with the infinite standard source domain. We may therefore take $d, e \geq 1$.

The composite $I \circ J$ interprets source numbers by suitable e-tuples of interpreted arithmetic elements, represented in the source by de numerical coordinates. The source-side bi-interpretation isomorphism provides a T-definable relation $\eta(a, \vec{u})$, with $\vec{u} \in D^e$, that is provably total for every source number a and is injective modulo the composite interpreted source equality. In particular, T proves

$$\forall a\, \exists \vec{u}\, \eta(a, \vec{u}), \qquad \eta(a, \vec{u}) \wedge \eta(a', \vec{u}\,') \wedge E_J^I(\vec{u}, \vec{u}\,') \longrightarrow a = a'. \tag{Source round trip}$$

Domain conditions are included in η. Here E_J^I denotes the interpretation under I of the equality relation used by J. No restrictions are placed on the quantifiers in η or the domain formulas. Only this source-side part of bi-interpretability will be used.

Fix an external positive integer k. Arithmetic A has primitive-recursive injective tupling for ke numerical inputs, with primitive-recursive projections; its decoding equations are theorems of A. Interpret that tupling under I. It gives a provably total formula-defined coding relation from $(D/E)^{ke}$ to D/E, injective on its input tuples.

Define $R_k(a_1, \ldots, a_k, \vec{c})$ in T as follows: there exist η-representatives $\vec{u}_i \in D^e$ of all a_i, and the interpreted arithmetic tupling relation codes the ke arithmetic components of $(\vec{u}_1, \ldots, \vec{u}_k)$ by $\vec{c} \in D$. This is one first-order formula. Its matrix may contain arbitrary quantifiers. From the first clause of (Source round trip) and the totality of arithmetic tupling, ordinary finite existential reasoning proves

$$T \vdash \forall a_1 \cdots \forall a_k\, \exists c_1 \cdots \exists c_d\, R_k(\vec{a}, \vec{c}). \tag{Compression totality}$$

There is no interpreted-domain assumption on the input scalars a_i: they range over the entire source domain. No form of choice is required; only the finitely many witnesses in this fixed formula are introduced.

Suppose that $R_k(\vec{a}, \vec{c})$ and $R_k(\vec{a}\,', \vec{c})$ hold. The same output representative $\vec{c}$ represents the same arithmetic element, by reflexivity of E on D. Injectivity of interpreted arithmetic tupling therefore gives

componentwise E-equality of all corresponding arithmetic components of $\vec{u}_i$ and $\vec{u}'_i$. Interpretations respect equality, so reflexivity and congruence for the interpreted J-equality imply

$$E_J^I(\vec{u}_i, \vec{u}'_i) \qquad (1 \le i \le k).$$

The second clause of (Source round trip) gives $a_i = a'_i$ for every i. These are all legitimate interpreted first-order consequences, so T proves that the output fibers of R_k are disjoint in exactly the sense of (Disjoint fibers). Surjectivity of η, uniqueness of the chosen representatives, and uniqueness of the raw output tuples are not needed.

Now choose $k = d + 1$. Formula (Compression totality) is a provably total relation from k unrestricted source scalars to d source coordinates with disjoint output fibers. This contradicts Theorem 9.2. □

Corollary 9.5 (Provably inhabited finite parameter families)**.** *The exclusion also applies to a finite parameterized bi-interpretation when T proves that its parameter condition is inhabited and proves the required interpretation and source-round-trip facts uniformly under that condition.*

Proof. Let $\vec{p}$ be the finite tuple of source parameters, of length r, and let its condition be $\Theta(\vec{p})$. Assume $T \vdash \exists \vec{p}\, \Theta(\vec{p})$ and that all the preceding interpretation and isomorphism facts are proved uniformly under $\Theta(\vec{p})$. Collect any additional finite parameter choices needed for those facts into the same tuple and condition.

The preceding construction, with these parameters retained, proves the totality and disjoint-fiber properties of $R_{k,\vec{p}}$ under $\Theta(\vec{p})$. Introduce the parameters as part of the output:

$$\widehat{R}_k(\vec{a}, \vec{p}, \vec{c}) := \Theta(\vec{p}) \wedge R_{k,\vec{p}}(\vec{a}, \vec{c}).$$

Using the proved inhabitedness first, T proves the unconditional sentence

$$\forall \vec{a}\, \exists \vec{p}\, \exists \vec{c}\, \widehat{R}_k(\vec{a}, \vec{p}, \vec{c}).$$

An identical output pair $(\vec{p}, \vec{c})$ uses identical parameter values. The disjoint-fiber argument for $R_{k,\vec{p}}$ therefore still gives equality of the input tuples. The number of output coordinates is now $d + r$, so choose $k = d + r + 1$ and apply Theorem 9.2. No restriction on the quantifiers in Θ is needed: it is inside the output matrix, rather than an extra assumption of the existential-free axiom theory. □

An externally chosen, model-specific parameter condition that T does not prove to be inhabited is not covered by this extension. Fixed parameters for which the interpretation facts are proved without extra assumptions are already covered by Theorem 9.4.

The proof explains why arbitrary positive existential domain formulas do not rescue *bi*-interpretability: they occur inside the matrix R_k, while the source-side isomorphism removes every domain condition from its unrestricted source inputs. Finite witness extraction makes no syntactic restriction on that matrix.

The theorem is a completed negative result for the ordinary tied-seed candidate and for the stated class of its extensions. It does not provide a smaller positive axiomatization of iPRA, does not rule out equiconsistency alone, and does not claim that every possible axiomatization in the B language admits existential-free axioms. It also does not automatically extend to additional bounded numerical primitives: they preserve the weaker coordinate bound, but need not preserve the finite-profile invariant used here.

10 A small countermodel to an attractive logic merger

Another investigated reduction tries to replace equality transport and closure use by one first-order clause eliminator. Encoding an equation $s = t$ as a clause that sends $r = s$ to $r = t$ gives root replacement. It does not recover numerical congruence for free.

Proposition 10.1 (Root replacement does not replace congruence)**.** *Numerical computation equations, ordinary zero/successor cases, induction, reflexivity, and root replacement are insufficient to derive successor congruence for a primitive equation predicate.*

Proof. Use the standard domain $\mathbb{N}$ and standard numerical functions. Interpret the primitive equation predicate by the equivalence relation $\approx$ whose only nonsingleton class is $\{2, 3\}$. Every computation equation holds because both sides have the same ordinary value. Reflexivity and root replacement are sound because $\approx$ is an equivalence relation. Cases and induction are sound on the standard natural numbers for every interpreted formula. Nevertheless $2 \approx 3$ holds while $S2 \not\approx S3$, since $3 \not\approx 4$. Hence successor congruence is not derivable in the weakened presentation. □

This is a countermodel to a proposed rule set; it is not the equality interpretation used for the complete theory above. The complete theory uses actual equality. A restricted Leibniz clause $P(s) \to P(t)$, schematic in a formula context P, does merge the two eliminations, but it moves the work into context application, sorted formation, and higher-order substitution. No smaller complete checker for that route is established here.

11 What fixed countdown uniqueness actually determines

This independent observation isolates a remaining gap in the supplied scalar-countdown program. The source is `ipra_scalar_countdown_2026-11-09.zip`, member `theory/completion.tex`, subsections "The only induction-type rule and the reflection sentence" and "Uniqueness of a zero-start successor trajectory," in the same pinned ProveIt snapshot. That report proves its completion using an additional level-two faithfulness sentence. The following lemma needs no such sentence, but its conclusion is weaker than equality of counters.

Use the source's bouncing predecessor W and scalar countdown L, which satisfy

$$L(0, a) = a, \qquad L(Sn, a) = W(L(n, a)). \tag{Countdown}$$

The contextual rule U_W is

$$\frac{\Gamma \vdash g(Sz, \vec{p}) = W(g(z, \vec{p}))}{\Gamma \vdash g(t, \vec{p}) = L(t, g(0, \vec{p}))}, \tag{U_W}$$

where g is one fixed finite numerical template and z is fresh for the retained context, target term, and passive data. No function variable is quantified in this rule.

Lemma 11.1 (Every zero-start successor clock has the same countdown action)**.** *Suppose, in a retained context Γ, that a fixed term template f satisfies*

$$\Gamma \vdash f(0) = 0, \qquad \Gamma \vdash f(Sz) = S(f(z)),$$

with the ordinary eigenvariable condition. Then for every target term t and every parameter a,

$$\Gamma \vdash L(f(t), a) = L(t, a). \tag{Action}$$

Thus the full ordinary logical presentation proves the universal closure over a of this equality.

Proof. Choose the eigenvariable z fresh also for a and set $g(n) = L(f(n), a)$. The assumed successor equation and (Countdown) give

$$g(Sz) = L(S(f(z)), a) = W(L(f(z), a)) = W(g(z)).$$

The assumed initial equation gives $g(0) = L(0, a) = a$. Apply U_W to this fixed template and substitute the latter equation in its conclusion. This is exactly (Action). All passive parameters remain fixed throughout the rule instance. □

For example, if $A(n, a)$ is the source's initialized successor iterator, then $f(n) = A(n, 0)$ satisfies the hypotheses by its computation equations. Consequently

$$L(A(t, 0), a) = L(t, a)$$

already follows from U_W.

Corollary 11.2 (The missing faithfulness implication)**.** *If the countdown source additionally proves*

$$\forall x\, \forall y\, ((\forall a\, L(x, a) = L(y, a)) \longrightarrow x = y), \qquad \text{(Faithful action)}$$

then it derives $f(t) = t$ for every zero-start successor clock as above.

Proof. Instantiate (Faithful action) with $x = f(t)$ and $y = t$ and apply the universal closure of (Action). □

This corollary is conditional. No proof of (Faithful action) from U_W alone is supplied, and no equivalence with the source's stronger single-observation sentence

$$\forall a\, \forall b\, (L(b, S^2 a) = 2 \longrightarrow a = b)$$

is claimed here. Equality of all countdown observations is a statement about an action; it must not be silently replaced by actual equality of its counter indices. In particular, the displayed lemma does not remove the faithfulness axiom from the current countdown axiomatization and does not establish a new complete axiomatization of iPRA.

12 Validation and reproducibility

The delivered producer regenerates thirty named certificate examples, fourteen positive and sixteen negative. Running the compact and readable complete programs in ordinary and optimized Python, including fourteen malformed-input cases, gives **176 successful checks of the expected verdict**. Nine targeted controls exercise mandatory major proofs, eager predecessor validation, the literal-equation progress restriction, witness freshness, and propagation of the current scope into a major proof. All nine produce the intended distinction. These are recorded executions, not a proof of universal soundness and not a Lean/Rocq verification.

The positives include both computation equations, state-using and constant updates, reflexivity by transport, atomic progress, arbitrary term specialization, compound case introduction, nonvacuous witnesses, paired elimination, transport below a closure, and contextual induction. The negatives include a false equation, failed majors, forged witnesses, unallocated parameters, forbidden compound progress, malformed counters, unsupported used data, and incorrect public input shapes. The new fresh-witness regression checks a false standard-model statement rather than merely causing an arbitrary parser error.

From the extracted package root:

```
python3 reproduce.py
python3 code/checker.py < examples/I0_arbitrary_seed.json
python3 code/checker.py < examples/forged_witness_reject.json
python3 build.py
```

The negative example is expected to print `REJECT` and exit nonzero. `reproduce.py` checks the actual current source sizes, reruns the certificate validation, regenerates deterministic gzip, checks the reviewed frontier sources, and exercises the finite graph lemma used in the obstruction proof. It does not purport to certify the mathematical metatheorems by computation. To rerun the entire archive audit, supply the exact pinned proposal ZIP directory to `audit/audit_corpus.py`; the required hashes are already included. The much larger input archives themselves are not duplicated in this deliverable. The six relevant complete baseline sources are included byte-for-byte.

13 Further research questions

Which additional primitive escapes the complete obstruction? Theorem 9.4 closes the arbitrary relational bi-interpretation route for the specified ordinary tied-seed theory. A successful repair must change at least one of its hypotheses, for example by adding numerical operations that violate the finite-profile invariant or by supplying positive existence axioms that cannot be eliminated in the old language. The cost of those additions must be charged to a complete checker.

Section 8 supplies a direct numerical repair with an unbounded zero selector. A formula definition of that selector alone does not justify closing the old recursion and induction schemes under the new symbol. Bounded numerical extensions preserve the weaker term-interpretation obstruction, but need not preserve the finite-profile invariant used in the full bi-interpretation theorem. These are distinct remaining research questions. One-way relational interpretations on complicated domains are not excluded by the return-map argument.

A focused calculus with fewer proved-major cases. Simply deleting recursive verification of elimination majors is incomplete: equation induction currently introduces a progress closure and then eliminates it. A sound focused redesign would need a proved normalization theorem or an explicit local induction-clause generator, with the replacement machinery included in its checker size.

Faithfulness of the countdown action. Section 11 proves equality of all countdown observations for a successor clock. Can fixed countdown uniqueness alone force those observations to distinguish counters? The supplied completion uses a level-two observation axiom. Removing it remains a specific open proof obligation, not an available simplification.

Proof-assistant verification of the scalar bridge. The largest logical risk is the long internal predecessor and equality reflection construction, not the four-byte change in scope storage. A Lean or Rocq verification should state its domain, actual equality, staged descriptions, passive-parameter conventions, and both interpretation composites explicitly. It should avoid reintroducing general quantifier-free induction before the equality characteristic has been proved correct.

An improvement under more than one source metric. The next complete program should improve the relevant normalized coordinates under the same public contract, while keeping a readable implementation alongside it. Maximum gzip size should continue to be reported as descriptive metadata.

Removing the rejection interface only from a new program, or hiding a theorem table outside its counted source, would not establish the requested advance.

Inherited complete bridge: attribution and reading convention

The following five source fragments are copied unchanged from the constructed-computation archive in the pinned current corpus. Their exact member paths and hashes are in `evidence/inherited_bridge.json`. They preserve the source authors' earlier attributions to the scalar, forward-left, and progress-closure developments. The repository's MIT No Attribution license is included. Words such as "old" and "new" inside these unchanged fragments refer to their original reports, not to a novelty claim of this investigation.

The first fragment uses an equivalent clause presentation of assumptions. An ordinary assumption is a constant clause with a fixed computation theorem as its antecedent; conversely each fixed clause is represented by its derived implication and universal quantification. The main calculus's auxiliary Q is its zero-antecedent closure. Its unused-witness Eq/D representative was stated in Section 2. These interderivabilities connect the presentations rather than silently adding a logical rule.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{3}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{4}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{5}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0)=1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n)+2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{6}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{7}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (7) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{8}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (7); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{9}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (8), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{10}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{11}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (11). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (12) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (13) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{14}$$

The base is (13). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{15}$$

The first uses the comparator recurrence. The second uses (14) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{16}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{17}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (16) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

C From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix B, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

C.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{18}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{19}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{20}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{21}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{22}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (22) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

C.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{23}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{24}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{25}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (18). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (24); symmetry and (21) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{26}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{27}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{28}$$

To prove the range assertion, split (20). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (19) also give, for all u, v,

$$C(u, v, v) = v. \tag{29}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{30}$$

If $u = 1 = S0$, (25) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

C.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{31}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{32}$$

Using these equations and (22), we now derive

$$0 + y = y, \tag{33}$$
$$(Sx) + y = S(x + y), \tag{34}$$
$$d(Sx, Sy) = d(x, y), \tag{35}$$
$$d(Sa, a) = 1. \tag{36}$$

Each induction below has a single equation as its induction formula.

For (33), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (34), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S((Sx) + y) = S(S(x + y)) = S(x + Sy).$$

Combining (34) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{37}$$

Indeed, both sides are $S(y + r)$.

For (35), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (36). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (35) and the induction hypothesis.

C.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{38}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (33) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (29).

For the step, assume (38) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (38) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (37) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (38). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{39}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (38). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (36) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{40}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{41}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{42}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (27), which is equivalent to $d(Sa, b) = 1$ by (30), and then to $a = b$ by (39). The range assertion is (26) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (41) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{43}$$

is a theorem, not an extra hypothesis.

C.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{44}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{45}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{46}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{47}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{48}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (41)–(42). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (48) for every quantifier-free formula.

C.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (48), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at

zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

D Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

D.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k + 1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

D.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

D.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

E Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u,\vec{p})$, introduce $i_h(n,a,\vec{p})$ with the two equations

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

E.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem E.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt: draft collection*, pinned commit `3fb35a5f2c0cb506180bede7ee3b9b5f93d4bc9d`. https://github.com/VladimirReshetnikov/ProveIt/tree/3fb35a5f2c0cb506180bede7ee3b9b5f93d4bc9d/docs/drafts.

[2] *Numerical axioms as local assumptions*, `ipra_generated_assumptions_2026-11-10.zip` in [1]. Source of the generated-assumption arithmetic branch, sorted decoder, known-instance discharge, and named example producer.

[3] *Constructing computation sources*, `ipra_constructed_computation_2026-11-10.zip` in [1]. Immediate source of the five reproduced bridge appendices.

[4] *Transient seed* development, `ipra_transient_seed_2026-11-10.zip` in [1]. Source of the tied-seed B grammar and its inherited one-coordinate bound, especially `theory/obstruction.tex`.

[5] *Counter congruence* development, `ipra_counter_congruence_2026-11-10.zip` in [1]. The exact byte comparator is `ancillary/zero_product.py`.

[6] Inherited scalar predecessor construction, reproduced as `appendices/scalar_predecessor.tex` in [3], with its earlier scalar-development attribution.

[7] Inherited actual-equality reflection construction, reproduced as `appendices/equality_reflection.tex` in [3], with attribution to the forward-left development.

[8] Inherited pairing and primitive-recursion compiler, reproduced as `appendices/primitive_recursion.tex` in [3], with attribution to the progress-closure development.

[9] Michael Pinsker. *Clones containing all almost unary functions*. Algebra Universalis **51** (2004), 235–255. https://www.dmg.tuwien.ac.at/pinsker/papers/interval/interval.pdf. Section 1.1 supplies the established terminology; the finite-quotient unit argument in this report is given independently.

[10] Jeremy Avigad. *Interpreting classical theories in constructive ones*. Journal of Symbolic Logic **65**(4) (2000), 1785–1812. https://www.andrew.cmu.edu/user/avigad/Papers/interp.pdf.

[11] Jeremy Avigad. *Algebraic proofs of cut elimination*. Journal of Logic and Algebraic Programming **49** (2001), 15–30. https://www.andrew.cmu.edu/user/avigad/Papers/cutelim.pdf. Theorem 7.6 supplies intuitionistic first-order cut elimination. The finite-witness application and arithmetic obstruction in this report are derived separately.